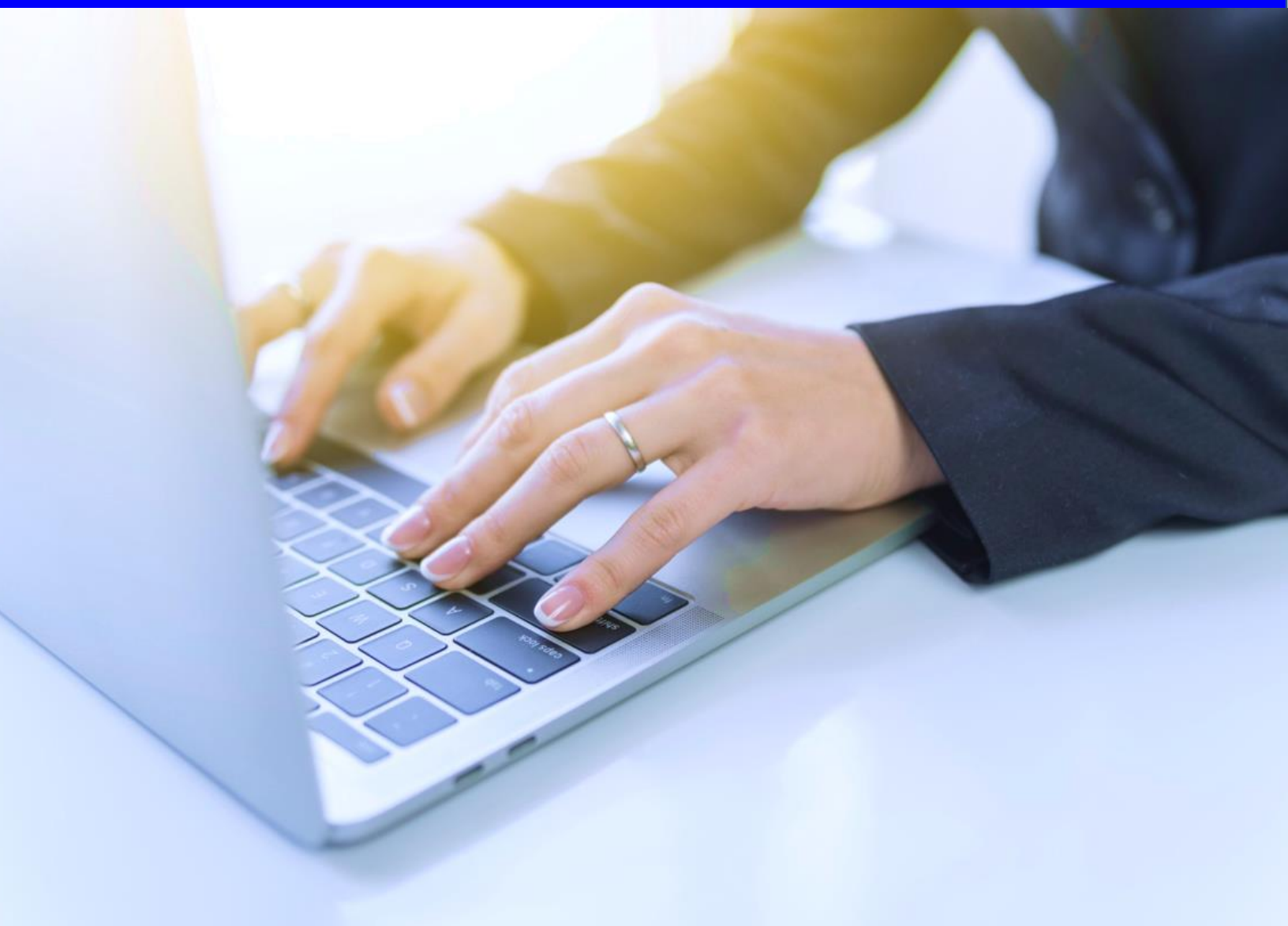


Nombre de la Asignatura:		PROGRAMACIÓN I
Unidad	Nº1	PRIMEROS PASOS CON C#



Consideraciones Previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Reconoce la herencia de clase e implementación de interfaces como parte del concepto de polimorfismo presente en la POO para la construcción de aplicaciones más robustas y escalables a nivel de código.
- Reconoce distintos tipos de colecciones como métodos de almacenamiento de datos en memoria para guardar información de forma temporal durante la ejecución de una aplicación.

SOBRE LAS FUENTES UTILIZADAS EN EL MATERIAL

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

Al momento de desarrollar una aplicación, uno de los puntos más importantes para los desarrolladores es mantener un orden a nivel de código para permitir su mantención, escalabilidad y reusabilidad. Existen diversas técnicas que permiten alcanzar estos objetivos, y una de esas es la programación orientada a objetos con principios como la herencia, la abstracción y polimorfismo.

Durante este y otros recursos, se verá cómo funciona la programación orientada a objetos en C#, como almacenar objetos dentro de colecciones de distinto tipo y como las interfaces permiten tener contratos dentro de las clases.

Ideas fuerza

- **Paradigma:** es un estilo de programación basado en las características del lenguaje de programación. Existen lenguajes de programación multiparadigma, permitiendo escribir código de distintas formas para lograr el mismo objetivo
- **Interface:** estructura especial utilizada para definir contratos, los cuales son implementados por una o más clases.
- **Contratos:** corresponde a la firma de un método o propiedad definida dentro de una clase de tipo interface.
- **ICollection:** es una interface utilizada en clases derivadas de ICollection dentro de C#. Contiene los métodos y props básicas que implementan las colecciones dentro de este lenguaje.

Índice

<i>Consideraciones Previas</i>	2
SOBRE LAS FUENTES UTILIZADAS EN EL MATERIAL.....	2
<i>Introducción</i>	3
<i>Ideas fuerza</i>	4
<i>Desarrollo</i>	6
1. Polimorfismo	6
a. Herencia.....	6
b. Interface.....	11
2. Colecciones	13
a. List.....	13
b. ArrayList.....	17

Desarrollo

1. Polimorfismo

Entre todas las características de C#, destaca que es un lenguaje orientado a objetos. Permite crear clases, las que luego pueden ser transformadas a objetos para poder almacenar información de forma estructurada.

La programación orientada a objetos se caracteriza por cuatro principios básicos: abstracción (abstraction), encapsulamiento (encapsulation), herencia (inheritance) y polimorfismo (polymorphism).

La **abstracción** permite crear clases para representar atributos y comportamientos relevantes que estén presentes en el sistema.

El **encapsulamiento** es un mecanismo utilizado para esconder el estado interno y funcional de un objeto, siendo estos solo accesibles por medio métodos públicos.

La herencia y polimorfismo serán vistos en profundidad a continuación:

a. Herencia

Al igual que en la vida real, en donde los hijos (por lo general) heredan de los padres, en la programación orientada a objeto ocurre algo parecido. La **herencia** permite que una clase pueda heredar comportamientos (métodos) y características (propiedades y atributos) desde otra clase. En este proceso participan dos entes: la clase padre y la clase hijo.

Por un lado, la clase hijo puede acceder a la información de la clase padre (métodos y props/atributos).

Por otro lado, la clase padre **no** puede acceder a la información de la clase hijo.

Para invocar un método o propiedad de la clase padre, se deberá utilizar la instrucción reservada `base`, la cual hará referencia a la clase padre desde la cual se hereda.

Ejemplo

En el siguiente ejemplo, se verán dos clases: Personal y Profesor. La clase Personal funcionará como clase padre y la clase Profesor como la clase hijo.

Imagen 1: Clase Personal actuando como clase padre

```
public class Personal
{
    public int Id { get; set; }
    public string? Nombre { get; set; }
    public string? Apellido { get; set; }

    protected string ObtenerInformacion()
    {
        return $"Id: {Id} - Nombre: {Nombre} {Apellido}";
    }
}
```

Fuente: Elaboración propia

Imagen 2: Clase Profesor heredando desde clase Personal

```
public class Profesor : Personal
{
    public Profesor() : base() {}

    public string MostrarInfoDesdeClaseProfesor()
    {
        return base.ObtenerInformacion();
    }
}
```

Fuente: Elaboración propia

Imagen 3: Creación de instancia de objeto Profesor

```
Profesor profe = new Profesor()  
{  
    Id = 1,  
    Nombre = "Albert",  
    Apellido = "Einstein"  
};  
  
profe.MostrarInfoDesdeClaseProfesor();
```

Fuente: Elaboración propia

En este ejemplo, la clase Profesor hereda tanto los métodos como las propiedades de la clase. Cabe destacar que el método ObtenerInformacion() presente en la clase Personal, es de tipo protected, por lo que, cualquier clase hija podrá acceder a este utilizando la instrucción reservada base.

A reflexionar

¿Una clase puede heredar comportamientos y características desde más de una clase? Si es así, ¿Cómo se hace la diferenciación entre las distintas clases padre con la instrucción reservada base?



El **polimorfismo** va muy ligado a la herencia. El polimorfismo establece dos aspectos:

- La clase padre puede crear instancias de la clase hija. Sin embargo, la clase hija no puede crear instancias de la clase padre.

Imagen 4: Creación instancia de objeto de clase Profesor mediante la clase Personal

```
Personal historia = new Profesor()  
{  
    Id = 2,  
    Nombre = "José",  
    Apellido = "Mourinho"  
};
```

Fuente: Elaboración propia

La imagen anterior muestra como la clase padre es capaz de crear instancias de objeto de la clase hija, siendo esto posible solo si la clase hija (Profesor) hereda desde la clase padre (Personal).

En caso de que se intentase realizar al revés, en donde sea la clase hija la que cree una instancia de objeto de la clase padre, se arrojaría un error en pantalla.

Imagen 5: Error lanzado al momento de crear instancia de objeto de clase Personal a partir de clase Profesor

```
Personal.Personal()

CS0266: Cannot implicitly convert type 'Herencia_Colecciones.Inheritance.Personal' to
'Herencia_Colecciones.Inheritance.Profesor'. An explicit conversion exists (are you missing a cast?)
```

Fuente: Elaboración propia

- Las clases padre definen métodos y las clases hijo pueden sobre escribirlos. Para que esto sea posible, el método de la clase padre deberá ser definido como virtual, mientras que para que el hijo pueda sobre escribir dicho método, deberá utilizar la palabra reservada override.

Imagen 6: Método virtual en clase Personal

```
public virtual string MetodoVirtual()
{
    return "Soy un método virtual desde la clase padre!";
}
```

Fuente: Elaboración propia

Imagen 7: Sobre escritura de método en clase Profesor

```
public override string MetodoVirtual()
{
    return "Soy un método virtual de la clase padre " +
           "que ha sido sobre escrito en la clase hijo!";
}
```

Fuente: Elaboración propia

b. Interface

Las interfaces o interface (no confundir con interfaz de usuario), son un tipo de estructuras que permiten definir contratos. Un contrato será todo método o propiedad que sea definida dentro de la estructura de la interface, los cuales además son obligatorios.

Dentro de una interface, los miembros que la compongan no deben tener asignado un modificador de acceso, ya que por defecto son considerados de tipo público, pero si pueden recibir parámetros de entrada y tener un tipo de dato como retorno (string, int, bool, etc) o simplemente ser de tipo void.

Imagen 8: Firma de método y propiedad dentro de interface

```
public interface IPersistente
{
    bool ObtenerPersonalPorId(int id);
    bool Estado { get; set; }
}
```

Fuente: Elaboración propia

Por convención, todas las estructuras de tipo interface deben anteponer una *i* latina mayúscula (I) antes del nombre real de la interface creada, como en es demostrado en el ejemplo anterior.

Toda clase que herede desde una interface deberá implementar de forma obligatoria los métodos y propiedades que hayan sido definidos dentro de este.

Imagen 9: Implementación de interface dentro clase Personal

```
public class Personal : IPersistente
{
    public int Id { get; set; }
    public string? Nombre { get; set; }
    public string? Apellido { get; set; }
    public bool Estado {
        get => throw new NotImplementedException();
        set => throw new NotImplementedException();
    }

    protected string ObtenerInformacion()
    {
        return $"Id: {Id} - Nombre: {Nombre} {Apellido}";
    }

    public virtual string MetodoVirtual()
    {
        return "Soy un método virtual desde la clase padre!";
    }

    public bool ObtenerPersonalPorId(int id)
    {
        //TODO code
        throw new NotImplementedException();
    }
}
```

Fuente: Elaboración propia

Tanto el método `ObtenerPersonalPorId(int id)` como la propiedad `Estado`, además de estar disponibles dentro de la clase `Personal`, lo estarán dentro de la clase `Profesor`, debido al proceso de herencia.

A reflexionar

¿Es posible establecer que un método pueda no ser sobre escrito? ¿Existe alguna instrucción especial?



2. Colecciones

En matemáticas existen los denominados conjuntos, que como su nombre lo indica, agrupa conjuntos de valores, los cuales pueden tener alguna característica similar o ser completamente distintos.

En informática también existen estructuras similares a los conjuntos que permiten almacenar valores de distinto tipo, denominadas colecciones (aunque también se les puede encontrar como matrices, listas o arrays).

En C# existen muchos tipos de colecciones (en serio, muchos tipos) dependiendo del uso que se le quiera otorgar en el código. Específicamente se verán dos tipos:

- List
- ArrayList

a. List

Las colecciones de tipo List o lista son creados por medio de la clase List<T>. Esta acepta genéricos, es decir, puede almacenar conjuntos de datos de cualquier tipo: int, string, boolean, e incluso otras listas (esto es conocido como matriz bidimensional). Sin embargo, una vez establecido el tipo de dato que recibirá, no se podrán almacenar valores de otros tipos de datos, a menos que el valor a agregar sea “casteado” o transformado a ese tipo de dato. Por ejemplo, un List<string> solo recibirá cadenas de texto. Si se intenta almacenar una prop o valor de tipo int, se arrojará un error, a menos que este valor sea transformado, en esta oportunidad, a string.

A cada registro almacenado dentro de una colección, se le asigna un valor posicional automático denominado índice. Este representa la posición en la que se encuentra un registro y siempre parten desde el cero.

Ejemplo:

Existe una línea de 5 sillas, en donde cada silla tiene un número posicional (o índice) asignado partiendo desde el 0 hasta el 4 (dando un total de 5 sillas) . Además, cada silla tiene un color distinto.

Imagen 10: Lista de sillas con índice asociado a la posición



Fuente: Elaboración propia

Si, por ejemplo, se quisiese buscar la silla naranja de la línea de sillas, entonces se podría utilizar su índice, el que corresponde a 3.

Imagen 11: Silla obtenida desde la lista por medio del índice posicional



Fuente: Elaboración propia

Si esto se llevara a código se vería así:

1. Creando la clase Silla

Imagen 12: Clase Silla con propiedad Color

```
public class Silla
{
    public string? Color { get; set; }
}
```

Fuente: Elaboración propia

2. Creando listado de objetos tipo Silla con clase List<T>

Imagen 13: Creación de listado de objetos

```

List<Silla> filaSillas = new List<Silla>();

Silla roja = new Silla() { Color = "Rojo" };
Silla amarilla = new Silla() { Color = "Amarilla" };
Silla verde = new Silla() { Color = "Verde" };
Silla naranja = new Silla() { Color = "Naranja" };
Silla azul = new Silla() { Color = "Azul" };

filaSillas.Add(roja);
filaSillas.Add(amarilla);
filaSillas.Add(verde);
filaSillas.Add(naranja);
filaSillas.Add(azul);

```

Fuente: Elaboración propia

3. Mostrando silla naranja (posición 3)

Imagen 14: Obtención del color de objeto Silla desde la lista utilizando índice posicional

```

Console.WriteLine(filaSillas[3].Color);

```

Fuente: Elaboración propia

List es una clase, y, por ende, posee comportamientos (métodos) y características (propiedades). En conjunto, permiten administrar no solo la matriz en sí, sino que también los registros que este almacena.

Algunas de los métodos más importantes son:

Tabla 1: Métodos clase List<T>

Método	Retorno	Descripción	Ejemplo
--------	---------	-------------	---------

Add(Object value)	int	agrega nuevo elemento al final del listado	colores.Add("rojo");
Clear()	void	elimina todos los elementos	colores.Clear();
Contains(Object value)	bool	determina si lista contiene el valor especificado	colores.Contains("rojo");
Insert(int i, Object value)	void	inserta un elemento el índice especificado	colores.Insert(5, "verde");
Remove(Object value)	void	remueve la primera ocurrencia en base al objeto especificado	colores.Remove("verde");
RemoveAt(int i)	void	remueve un elemento en base al índice especificado	colores.RemoveAt(4);

Fuente: Elaboración propia

Por el lado de las propiedades, es posible destacar un par de estas:

Tabla 2: Propiedades clase List<T>

Propiedad	Retorno	Descripción	Ejemplo
Count	int	Obtiene el número de elementos en la lista genérica	colores.Count;
Item[int i]	object	Obtiene un elemento según el índice entregado	colores[2].Color;

Fuente: Elaboración propia

b. ArrayList

Al contrario de las colecciones de tipo List<T> que pueden recibir cualquier tipo de dato, los ArrayList son colecciones no genéricas, por lo que suelen utilizarse para almacenar elementos cuya estructura, tamaño o tipo de dato es desconocido.

Actualmente, no es recomendado utilizar ArrayList, debido a problemas de performance que puedan presentarse al ser implementados. Sin embargo, si puede ser implementado en casos especiales, como en el caso expuesto anteriormente.

Al igual que las colecciones de tipo List<T>, ArrayList también es una clase y también tiene métodos y propiedades. Algunos de estos métodos son (para los siguientes ejemplos se creó una instancia de la clase ArrayList llamada colores):

Tabla 3: Métodos clase ArrayList<T>

Método	Retorno	Descripción	Ejemplo
Add(Object value)	int	agrega un nuevo objeto o valor al final de la lista	colores.Add("rojo")
AddRange()	void	añade una colección de datos al final de la lista	colores.Add(otrasSillas);
Clear()	void	elimina todos los elementos de la	colores.Clear();
Contains()	bool	determina si el elemento está presente en el listado	colores.Contains(56);
Insert(int i, Object value)	void	inserta nuevo elemento en el índice especificado	colores.Insert(7, papel);
Remove()	void	remueve la primera ocurrencia en base al objeto especificado	int obj = 56; colores.Remove(obj);
RemoveAt()	void	remueve un elemento en base al índice especificado	colores.RemoveAt(12);
Sort()	void	ordena los elementos dentro de un listado	colores.Sort();
ToArray()	object[]	retorna la colección de datos como un nuevo objeto de tipo array	colores.ToArray();
ToString()	string	retorna una cadena de texto que representa el listado (esto no lista cada uno de los elementos contenidos en la colección, solo muestra información de la clase)	colores.ToString();

Fuente: Elaboración propia

En cuanto a propiedades:

Tabla 4: Propiedades clase ArrayList<T>

Propiedad	Retorno	Descripción	Ejemplo
-----------	---------	-------------	---------

Capacity	int	establece u obtiene el número de elementos que puede contener el arraylist	colores.Capacity;
Count	int	Obtiene el número de elementos en la lista genérica	colores.Count;
Item[int i]	object	Obtiene un elemento según el índice entregado	int i = 4; colores.[i];

Fuente: Elaboración propia

Conclusión

C# es un lenguaje de programación multifacético. Es conocida su capacidad de poder construir un sinnúmero de tipos de aplicaciones, siendo este proceso facilitado por las distintas formas en la que permite crear el código.

Uno de estos caminos es la programación orientada a objetos. Este es un paradigma de programación que permite trabajar con clases e instancias de objetos para poder separar roles y encapsular las funcionalidades.

La programación orientada a objetos se centra en cuatro principios: la abstracción, encapsulamiento, la herencia y el polimorfismo.

Por ejemplo, la herencia establece que un objeto puede heredar comportamientos y características desde otro objeto, lo cual va de la mano con el polimorfismo, que establece que las clases hijas pueden realizar sus propias implementaciones de los métodos y propiedades de la clase, además de ser tratadas como tal.

Todos estos objetos que se pueden ir generando a lo largo del código, es probable que deban ser almacenados en memoria. Para este cometido se pueden utilizar las colecciones, que, más conocidas como listas o arrays, permiten agrupar objetos que tengan la misma estructura o que posean estructura desconocida.

Bibliografía

Microsoft (21 de septiembre 2022) **Overview of classes, structs, and records in C#**. Recuperado el 03 de junio de 2023 de <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/>

Microsoft (16 de febrero 2022) **Inheritance – derive types to create more specialized behavior**. Recuperado el 03 de junio de 2023 de <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance>

Microsoft (31 de enero de 2023) **Polymorphism**. Recuperad el 3 de junio de 2023 de <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>

Microsoft (17 de marzo de 2023) **Interfaces - define behavior for multiple types**. Recuperado el 03 de junio de 2023 de <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/interfaces>

Microsoft (sin información) **List<T> Class**. Recuperado el 03 de mayo de 2023 de <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-7.0>

Microsoft (sin información) **ArrayList Class**. Recuperado el 03 de mayo de 2023 de <https://learn.microsoft.com/en-us/dotnet/api/system.collections.arraylist?view=net-7.0>

